

Actividad 04 - Sumador Nibble y Sumador 8 bits

Introducción

Roberto Hernández Zúñiga 14 02 26 scribe

Actividad 04 - Aritmética computacional

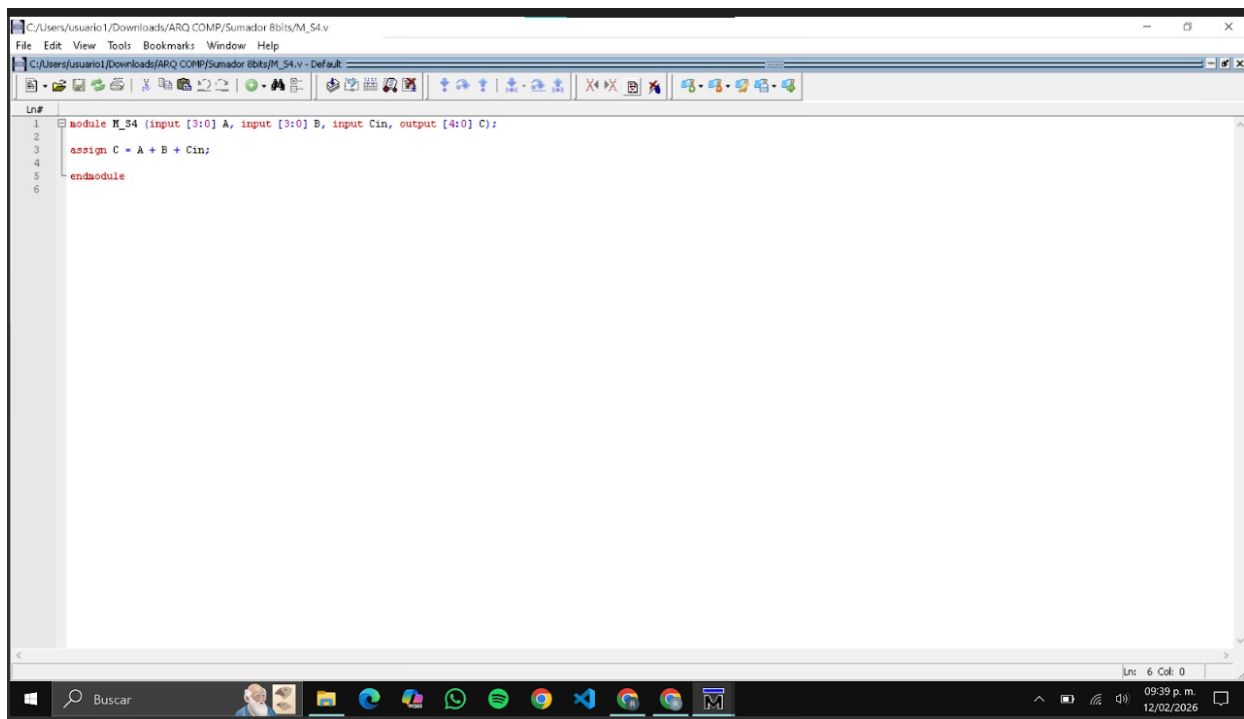
- **Complemento a 1.**
Es un método utilizado para representar números negativos en sistemas binarios. Este método se aplica a números binarios enteros y se utiliza principalmente en sistemas digitales y aritmética computacional.
Tiene la ventaja de simplificar las operaciones aritméticas en sistemas binarios, para obtener el complemento a 1 de un número binario, simplemente se invierten todos los bits de su representación binario.
- **Complemento a 2.**
Es otro sistema utilizado para representar números negativos en binario, pero corrige los problemas del complemento a 1 y es el más comúnmente utilizado en la mayoría de las computadoras.
Para obtener el complemento a 2 de un número binario se toma el complemento a 1 del número y se le suma 1 al resultado.
- **Punto Flotante.**
El sistema de punto flotante se utiliza para representar números reales en la computadora.
Se descompone en tres partes:
 1. **Signo:** Un bit que indica si el número es positivo o negativo
 2. **Exponente:** Un número que determina la magnitud del número
 3. **Manísa:** Parte significativa del número, representa los valores reales

Objetivos

Para esta practica el objetivo fue crear 2 cosas que son el sumador de 4 bits y el sumador de 8 bits en verilog, pero el objetivo era hacerlo de 2 formas distintas, la primera de ellas era usando un half adder, ese half adder instanciarlo para crear un full adder, ese full adder instanciarlo para hacer el sumador de 4 bits y ese sumador igualmente instanciarlo para hacer finalmente el sumador de 8 bits. Mientras que la otra forma era por sumas comportamental, iniciamos del sumador de 4 bits ya funcional y de ahí solo instanciamos para crear el de 8 bits, aunque también tuvimos que hacer el testbench de estos 2 sumadores

Desarrollo

Primero que nada, hice el sumador de 4 bits, tuve que declarar las entradas A y B con la capacidad de 4 bits cada una, el acarreo de entrada y una salida C con capacidad de 5 bits, ya que en el peor de los casos la suma puede llegar a ser de 5 bits, sumo estos elementos y se los asigno a la salida.

A screenshot of a Verilog code editor window. The title bar shows the file path: C:/Users/usuario1/Downloads/ARQ COMP/Sumador 8bits/M_54.v. The menu bar includes File, Edit, View, Tools, Bookmarks, Window, and Help. The toolbar contains various icons for file operations and editing. The code editor area shows the following Verilog code:

```
1 module M_54 (input [3:0] A, input [3:0] B, input Cin, output [4:0] C);  
2  
3     assign C = A + B + Cin;  
4  
5 endmodule  
6
```

The status bar at the bottom indicates line 6, column 0. The Windows taskbar is visible at the very bottom with the search bar and several application icons.

Después de esto procedo a crear el sumador de 8 bits, aquí lo que hago es declarar las entradas A y B, pero estas de 8 bits cada una, y la salida C de 9 bits igual por si llegara a ocurrir el peor de los casos.

Ahora es cuando instancio 2 veces el sumador de 4 bits, en la primera instancia le paso a A y B solo 4 bits, el acarreo de entrada para este lo mando a 0 y la salida es simplemente la suma de A y B, el detalle es que esta salida va conectada a 2 partes, la primera es a C y a esta parte le van los primeros 4 bits, la otra parte es al acarreo de entrada del siguiente sumador de 4 bits, y a esta solo le llega el 5to bit, esto lo hago a través de un cable ya declarado.

Para la segunda instancia tengo igual las entradas A y B, pero estas empiezan del bit 5 y terminan hasta el bit 8; el acarreo de este sumador es el bit de la salida del anterior, así que simplemente se hace la suma y la salida de este sumador va a C, pero ahora este abarca del bit 5 al 9.

```
1 module M_S8 (input [7:0] A, input [7:0] B, output [8:0] C);
2
3
4 wire c1;
5
6 M_S4 sum (.A[A[3:0]], .B[B[3:0]], .Cin(1'b0), .C({c1,C[3:0]});
7 M_S4 sum2 (.A[A[7:4]], .B[B[7:4]], .Cin(c1), .C(C[8:4]));
8
9 endmodule
```

Y ahora solo quedan los testbench de los 2 sumadores y fue el mismo proceso para ambos, simplemente es instanciar el sumador poniendo prácticamente las mismas variables, y ahora solo se abre un apartado en el que le das un valor a A y otro a B, adjunto como se ve el código de los testbench junto a su simulación.

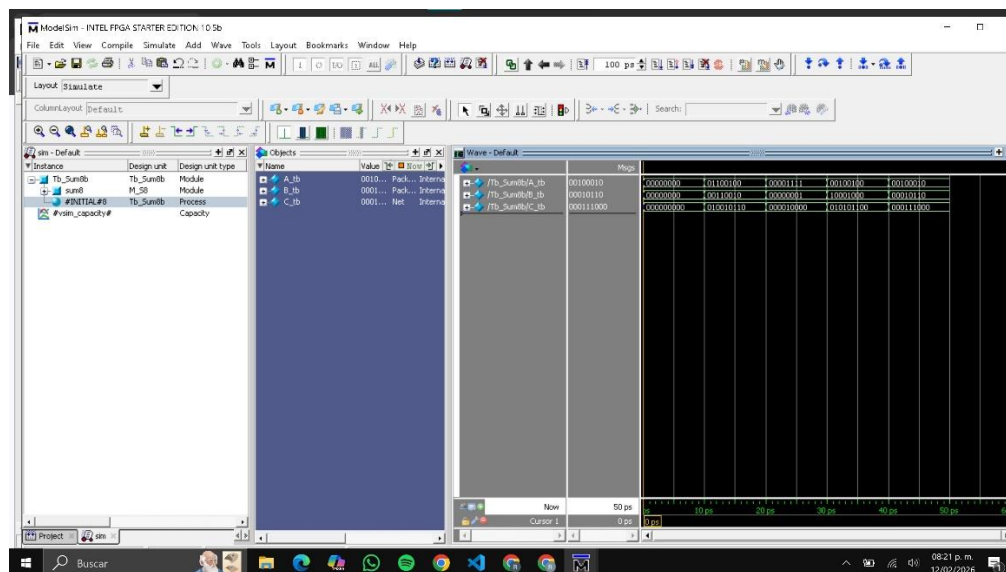
```
1 module Tb_Sum4;
2 reg [3:0] A_tb;
3 reg [3:0] B_tb;
4 reg Cin_tb;
5 wire [4:0] C_tb;
6
7 M_S4 sum4 (.A(A_tb), .B(B_tb), .Cin(Cin_tb), .C(C_tb));
8
9 initial begin
10     $monitor("Tiempo: %0t | A: %d + B: %d + Cin: %d = Resultado: %d", $time, A_tb, B_tb, Cin_tb, C_tb);
11     A_tb = 4'd5; B_tb = 4'd3; Cin_tb = 0; #10; // 5 + 3 = 8
12     A_tb = 4'd10; B_tb = 4'd2; Cin_tb = 1; #10; // 10 + 2 + 1 = 13
13     A_tb = 4'hf; B_tb = 4'hf; Cin_tb = 1; #10; // 15 + 15 + 1 = 31
14     A_tb = 0; B_tb = 0; Cin_tb = 0; #10;
15     A_tb = 4'd24; B_tb = 4'd6; Cin_tb = 0; #10;
16     $finish;
17 end
18 endmodule
```

```

C:/Users/usuario1/Downloads/ARQ COMP/Sumador 8bits/Tb_Sum8b.v
File Edit View Tools Bookmarks Window Help
C:/Users/usuario1/Downloads/ARQ COMP/Sumador 8bits/Tb_Sum8b.v - Default

Ln#
1 module Tb_Sum8b;
2     reg [7:0] A_tb;
3     reg [7:0] B_tb;
4     wire [8:0] C_tb;
5
6     M_S8 sum8 (.A(A_tb),.B(B_tb),.C(C_tb));
7
8     initial begin
9
10        $monitor("A=%d | B=%d | Suma=%d", A_tb, B_tb, C_tb);
11
12        A_tb = 8'd0; B_tb = 8'd0; #10;
13
14        A_tb = 8'd100; B_tb = 8'd50; #10;
15
16        A_tb = 8'd15; B_tb = 8'd1; #10;
17
18        A_tb = 8'hd24; B_tb = 8'h88; #10;
19
20        A_tb = 8'd34; B_tb = 8'h16; #10;
21
22        $finish;
23    end
24 endmodule
25

```



Conclusión

La verdad esta practica si estuvo mas difícil, siento que estamos avanzando rápido y se me hizo satisfactorio poder crear estos códigos a base de puro pensar y retroalimentar.

Referencias

<https://es.khanacademy.org>

<https://www.geeksforgeeks.org>

<https://www.tutorialspoint.com>

